LAB EXPERIMENTS – DAY 2

KNOWLEDGE ENGINEERING AND DATA SCIENCE

Name: D. Meghana **Course Name: Knowledge Engineering and Data Science**

Reg No: 192372276 **Course Code: DSA1604**

EXPERIMENT 1:RULE-BASED EXPERT SYSTEM FOR NETWORK TROUBLESHOOTING

1. Aim

To develop a rule-based expert system that identifies common network problems and suggests suitable solutions.

2. Software Required

- Python 3.13
- Python IDLE

3. Algorithm

Start the program.

Ask whether the internet is working.

If it is not working, check the Wi-Fi connection.

If Wi-Fi is disconnected, display the problem and solution.

Otherwise, check the router and suggest a suitable solution.

If the internet is working, check whether it is slow.

Display the result and stop.

4. Program

```
print("RULE-BASED EXPERT SYSTEM FOR NETWORK TROUBLESHOOTING")
internet = input("Is the internet working? (yes/no): ").lower()
if internet == "no":
    wifi = input("Is Wi-Fi connected? (yes/no): ").lower()
    if wifi == "no":
        print("Problem: Wi-Fi is disconnected.")
        print("Solution: Connect to Wi-Fi.")
    else:
        router = input("Is the router working? (yes/no): ").lower()
        if router == "no":
            print("Problem: Router failure.")
            print("Solution: Restart the router.")
        else:
            print("Problem: Possible ISP issue.")
            print("Solution: Contact your Internet Service Provider.")
```

```
else:
    speed = input("Is the internet slow? (yes/no): ").lower()
    if speed == "yes":
        print("Problem: Slow network connection.")
        print("Solution: Reduce network usage or restart the router.")
    else:
        print("Network is working properly.")
```

5. Input

Internet working: no; Wi-Fi connected: no.

6. Output

```
Python 3.13.1 (tags/v3.13.1:0671451, Dec  3 2024, 19:06:28) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
=== RESTART: C:/Users/HP/AppData/Local/Programs/Python/Python313/kE EXP 4.py ===
RULE-BASED EXPERT SYSTEM FOR NETWORK TROUBLESHOOTING
Is the internet working? (yes/no): NO
Is Wi-Fi connected? (yes/no): NO
Problem: Wi-Fi is disconnected.
Solution: Connect to Wi-Fi.
>>>
```

EXPERIMENT 2: SEMANTIC NETWORK USING PYTHON

1. Aim

To represent relationships between network devices using a semantic network.

2. Software Required

- Python 3.13
- Python IDLE

3. Algorithm

Start the program.

Create a dictionary of network devices and their connections.

Visit each device and its connections.

Display every relationship using an arrow.

Display a success message and stop.

4. Program

```
print("SEMANTIC NETWORK USING PYTHON")
network = {
    "Computer": ["Switch", "Printer"],
    "Switch": ["Router"],
    "Router": ["Internet", "Firewall"],
    "Printer": [],
    "Internet": [],
    "Firewall": []
}
print("\nSemantic Network Relationships:\n")
for device, connections in network.items():
    for connection in connections:
        print(device, "--->", connection)
print("\nSemantic Network Created Successfully!")
```

5. Input

No keyboard input is required. Devices and relationships are defined in the program.

6. Output

```
>>>
= RESTART: C:/Users/HP/AppData/Local/Programs/Python/Python313/KP EXP (5).py
SEMANTIC NETWORK USING PYTHON

Semantic Network Relationships:

Computer ---> Switch
Computer ---> Printer
Switch ---> Router
Router ---> Internet
Router ---> Firewall

Semantic Network Created Successfully!
>>>
```

EXPERIMENT 3: PRODUCTION RULES AND FORWARD CHAINING

1. Aim

To use production rules and forward chaining to derive new facts from known facts.

2. Software Required

- Python 3.13
- Python IDLE

3. Algorithm

Start the program.

Define initial facts and production rules.

Check whether each rule's conditions match the known facts.

If the conditions match and the result is new, add it to the facts.

Repeat until no new facts are added.

Display the final facts and stop.

4. Program

```
print("PRODUCTION RULES AND FORWARD CHAINING")
facts = {"fever", "cough"}
rules = [
    ({"fever", "cough"}, "possible_flu"),
    ({"possible_flu"}, "consult_doctor"),
    ({"network_down", "router_off"}, "restart_router"),
    ({"wifi_disconnected"}, "connect_wifi")
]
print("Initial Facts:", facts)
changed = True
while changed:
    changed = False
    for conditions, result in rules:
        if conditions.issubset(facts) and result not in facts:
            facts.add(result)
            print("New Fact:", result)
            changed = True
print("Final Facts:", facts)
```

5. Input

Initial facts: "fever" and "cough", defined in the program.

6. Output

```
========== RESTART: C:/Users/HP/AppData/Local/Programs/Python/Python313/KE EXP 6.py =======
PRODUCTION RULES AND FORWARD CHAINING
Initial Facts: {'fever', 'cough'}
New Fact: possible_flu
New Fact: consult_doctor
Final Facts: {'fever', 'cough', 'possible_flu', 'consult_doctor'}
>>
```